

React Hooks, Reconciliation & Fiber — Q&A Notes

React Hooks, Reconciliation & Fiber

A Question & Answer Walkthrough

1. What are Hooks in React?

React Hooks are functions that let you use state and other React features inside function components, without needing to write a class component.

Before hooks (pre-2019), only class components could hold state or use lifecycle methods like `componentDidMount`. Function components were “dumb” — just props in, JSX out. Hooks changed that.

The core hooks

useState — adds local state to a function component:

```
const [count, setCount] = useState(0);
```

useEffect — runs side effects (data fetching, subscriptions, DOM manipulation) after render, and can clean up after itself:

```
useEffect(() => {  
  document.title = `Count: ${count}`;  
}, [count]); // re-runs only when count changes
```

useContext — reads a value from React context without wrapping the component in a `Consumer`:

```
const theme = useContext(ThemeContext);
```

useRef — holds a mutable value that persists across renders without triggering a re-render (commonly used to reference DOM elements):

```
const inputRef = useRef(null);
```

useMemo / useCallback — memoize expensive calculations or function references to avoid unnecessary recalculation/re-renders.

useReducer — like `useState` but for more complex state logic, similar to Redux-style reducers.

The rules of hooks

1. **Only call hooks at the top level** — not inside loops, conditions, or nested functions. React relies on call order to keep track of each hook's state between renders.
2. **Only call hooks from React function components or custom hooks** — not regular JS functions.

Custom hooks

You can build your own hooks by combining existing ones — a custom hook is just a function starting with `use` that calls other hooks inside it. This is the main way to share stateful logic between components.

```
function useWindowWidth() {
  const [width, setWidth] = useState(window.innerWidth);
  useEffect(() => {
    const handler = () => setWidth(window.innerWidth);
    window.addEventListener('resize', handler);
    return () => window.removeEventListener('resize', handler);
  }, []);
  return width;
}
```

2. `useState` — What does “giving your component memory” mean?

Without memory: normally, when a function runs, any variable inside it disappears once the function finishes.

```
function Counter() {
  let count = 0; // a normal variable

  function handleClick() {
    count = count + 1;
    console.log(count);
  }

  return <button onClick={handleClick}>Click me</button>;
}
```

If you click this button, *count* *does* go up in the console (1, 2, 3...) — but the button on the screen never updates to show it. Why? Because React only updates what’s on screen when it re-renders the component, and a plain variable like `count` gets reset back to 0 every time the component re-renders. It has no memory between renders.

With `useState`, React remembers the value for you, outside the function, even as the component re-renders over and over:

```
function Counter() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
  }

  return <button onClick={handleClick}>Clicked {count}
times</button>;
}
```

Calling `setCount(...)` does two things:

1. Tells React “remember this new value from now on.”
2. Tells React “please re-render this component so the screen matches.”

So next time the component runs, `useState(0)` doesn’t actually give you 0 again — it gives you back whatever value was last saved (1, then 2, then 3...). React is storing that value somewhere outside the function, tied to that specific component, so it survives across re-renders.

That “surviving across re-renders” is what **memory** means here. A normal variable forgets everything each time the function runs again. `useState` remembers.

3. What is Reconciliation? When does it happen?

Reconciliation is the process React uses to figure out **what actually changed** on the screen, so it only updates those specific parts instead of rebuilding the whole page every time.

Directly changing the real DOM (the actual HTML on screen) is slow. So React keeps a lightweight copy of the UI in memory called the **virtual DOM** — a JavaScript object describing what the UI should look like. When something changes, React:

1. Builds a new virtual DOM tree (what the UI *should* look like now)
2. Compares it to the previous virtual DOM tree (what it looked like *before*)
3. Figures out the minimal set of differences (“this text changed”, “this button’s color changed”)
4. Updates *only* those specific parts in the real DOM

That comparison step — figuring out the differences between old and new — is **reconciliation**, also called “diffing.”

When does it happen?

- **Initial render (project first starts):** There’s no “previous” tree yet, so React just builds the whole virtual DOM from scratch and puts everything on screen. This is called “mounting,” not reconciliation — there’s nothing to compare against yet.
- **Every re-render after that:** This is when reconciliation kicks in. Anytime something changes — you call `setCount(...)`, a prop changes, a parent re-renders — React builds a new virtual DOM tree and *compares* it to the last one. That comparison is reconciliation.

So: reconciliation happens **every time your component re-renders**, not just once at the start.

```
function Counter() {  
  const [count, setCount] = useState(0);  
  return <button onClick={() => setCount(count + 1)}>Clicked {count}  
times</button>;  
}
```

Every click calls `setCount`, which triggers a re-render, which triggers reconciliation: React compares “Clicked 3 times” (old) to “Clicked 4 times” (new), sees only the number changed, and updates *just that text* in the real DOM — it doesn’t recreate the whole button.

4. Reconciliation versus Rendering

Rendering

Rendering is when React calls your component function to figure out what it *should* look like. It’s the “describe the UI” step.

```
function Counter() {  
  const [count, setCount] = useState(0);
```

```

    console.log("rendering!"); // this runs every render
    return <button onClick={() => setCount(count + 1)}>Clicked {count}
times</button>;
  }

```

Every time `setCount` is called, React re-runs this function from top to bottom. That's a **render**. The output is just a description (a virtual DOM tree) — nothing has touched the actual screen yet.

Reconciliation

Reconciliation happens right after rendering. It's React comparing the *new* description to the *old* description to figure out what actually changed.

Commit

After reconciliation figures out the differences, React actually applies those changes to the real DOM. This is called the **commit** phase. This is the only step that touches the actual screen.

Putting it in order

Step	What happens	Analogy
1. Render	Component function runs, produces a new virtual DOM description	Drawing a new blueprint
2. Reconciliation	React compares new blueprint to old blueprint, finds differences	Comparing old blueprint vs new blueprint
3. Commit	React updates only the changed parts in the real DOM	Construction crew makes only the needed repairs

Key distinction:

- **Rendering** = "what should the UI look like now?" (runs your component function)
- **Reconciliation** = "what's different from before?" (compares two virtual DOM trees)
- **Commit** = "make the real DOM match" (touches the actual screen)

Rendering can happen without anything changing on screen at all — if React renders and reconciliation finds *zero* differences, the commit step does nothing, and the real DOM is untouched. That's a common optimization target: avoiding unnecessary renders so React isn't wasting time doing comparisons that lead nowhere.

5. Scheduling

What scheduling is

Scheduling is React deciding *when* to actually do the render work — not everything happens instantly the moment you call `setCount`.

When you call `setState`, React doesn't necessarily drop everything and render right away. It **schedules** that render to happen, and decides its priority compared to other work that might also need doing.

Why this is needed

Imagine a component tree that's huge — thousands of elements. If a user types into a search box (which should feel instant) while React is also busy rendering a large list update (which is less urgent), you don't want the typing to feel laggy just because React is stuck finishing the big render first.

So React needs a way to say: *"this update is urgent, do it now — that other update can wait a few milliseconds."* That decision-making is scheduling.

How it fits into the sequence

Step	What happens
0. Schedule	React receives the update request (e.g. <code>setCount</code> was called) and decides <i>when/in what order</i> to process it, based on priority
1. Render	Component function runs, produces new virtual DOM
2. Reconciliation	Compare new tree to old tree, find differences
3. Commit	Apply the differences to the real DOM

A concrete example

```
function App() {
  const [text, setText] = useState("");
  const [items, setItems] = useState(bigListOfThings);

  return (
    <>
      <input value={text} onChange={(e) => setText(e.target.value)} />
      <BigExpensiveList items={items} />
    </>
  );
}
```

If typing in the input and updating the big list both get triggered around the same time, React's scheduler can prioritize the typing update (marked "urgent" since it's direct user input) and delay the list re-render slightly, so the typing feels smooth instead of janky.

This capability comes from the **Fiber architecture** (React's internal engine, since React 16) — it broke rendering work into small units that can be paused, prioritized, and resumed, instead of one big blocking process. Newer features like `startTransition` and `useDeferredValue` are direct ways *you* can tell the scheduler "this update isn't urgent, feel free to deprioritize it."

In one line: scheduling = React deciding *when* and *in what order* to do render work, so urgent updates (like typing) don't get blocked by expensive ones (like re-rendering a huge list).

6. What is a Fiber?

The problem Fiber solves

Before React 16, once React started rendering, it couldn't stop until it walked the *entire* component tree. If your app had a huge tree, this walk could take long enough to block the browser — meaning even something as simple as typing in a text box, or a CSS animation, could visibly freeze mid-update. There was no way to pause and say “let something more urgent go first.”

Fiber is the rewrite of React's internals (introduced in React 16) that fixes this. It restructures how React walks the component tree so that rendering work can be **paused, resumed, reused, or thrown away** — instead of being one uninterruptible block.

So what actually is a fiber?

A **fiber** is a plain JavaScript object that represents a single unit of work — essentially, one node in your component tree (one component, one DOM element).

Instead of just calling functions and returning results directly, React builds a **fiber tree** — a linked-list-like structure where each fiber holds:

- what component it represents
- its props and state
- a pointer to its parent, its first child, and its next sibling
- what work needs to be done on it (create, update, delete)

Because it's a linked structure with pointers (not a normal call stack), React can stop in the middle of walking this tree, remember exactly where it left off, go do something more urgent, and come back later to resume from that exact fiber. A regular function call stack can't do that — once you're deep inside nested function calls, you can't pause and jump out.

How this connects to scheduling and reconciliation

Fiber is what makes render and reconciliation interruptible. Rendering and reconciliation happen fiber-by-fiber, and React can pause between processing individual fibers to check “is there something more urgent to do first?” That pause-and-resume capability is exactly what the scheduler relies on.

The **commit** phase (actually touching the real DOM) is *not* interruptible — once React starts applying changes to the screen, it finishes that in one go, so the user never sees a half-updated UI.

Simple mental model

Think of the old pre-Fiber approach like reading a book straight through without bookmarks — if the phone rings, you either finish the page or lose your place. Fiber is like reading with a bookmark on every single line — you can stop after any line, deal with something urgent, and come right back to that exact spot.

In one line: a fiber is a JavaScript object representing one unit of rendering work for one component, structured so React can pause, prioritize, and resume rendering instead of doing it all in one uninterruptible block.

7. type and key

These two are central to how reconciliation decides what to keep, update, or throw away.

type

Every React element (and every fiber) has a type — it says **what kind of thing this is**.

```
<button>Click me</button> // type: "button" (a string, for HTML tags)
<Counter />                // type: Counter (a function/class, for components)
```

When React compares the old tree to the new tree during reconciliation, the **first thing it checks is type**:

- **Same type?** React assumes it's the same thing, just possibly updated — it reuses the existing DOM node/component instance and just updates its props/content.
- **Different type?** React throws away the old thing completely (destroys it, loses its state) and creates a brand new one from scratch.

Example:

```
// Before
<button>Clicked 3 times</button>

// After
<button>Clicked 4 times</button>
```

Same type ("button") → React just updates the text, keeps the same real DOM button element.

```
// Before
<button>Clicked 3 times</button>

// After
<input value="hello" />
```

Different type ("button" vs "input") → React deletes the old button entirely and creates a fresh input. No state is preserved.

key

key matters specifically for **lists** — when you're rendering multiple similar items with `.map()`.

```
{items.map(item => <li key={item.id}>{item.name}</li>)}
```

Without a key, React compares list items **by position** (index 0 to index 0, index 1 to index 1, etc.). This causes problems if the list gets reordered, or an item gets removed from the middle — React might think the *content* changed at each position, when really the *order* changed, causing unnecessary re-renders or even mixed-up state (e.g., text typed into the wrong input after a reorder).

key gives React a stable identity for each item, independent of position. Now React can say: "item with key '42' moved from position 2 to position 0 — same item, just reorder it, don't recreate it."

Concrete example of the problem:

```
// Without stable keys - using array index as key (bad practice)
```

```
const list = ["Apple", "Banana", "Cherry"];
list.map((item, index) => <li key={index}>{item}</li>);
```

If you now remove “Apple” from the front:

```
["Banana", "Cherry"]
```

React sees: position 0 was “Apple” (key=0), now it’s “Banana” (key=0) — same key! React thinks the *content* of that same item just changed from “Apple” to “Banana”, instead of realizing “Apple” was deleted and the others shifted up. If those list items held internal state (like a checked checkbox), that state would now incorrectly stick to the wrong item.

Fixed with a real, stable ID:

```
list.map(item => <li key={item.id}>{item.name}</li>);
```

Now React can correctly track: “the item with id apple-1 is gone” and “banana and cherry just moved up” — without confusing their identities.

How type and key work together in reconciliation

When React compares two lists during reconciliation, it matches up old and new elements using **both**:

“Is this the same type *and* the same key?” → if yes, reuse and just update it. If no → destroy old, create new.

In one line:

- type tells React *what kind* of element/component this is (used to decide reuse vs. recreate).
- key tells React *which specific instance* this is among siblings (used to correctly track identity when lists change order).

Summary Table

Concept	One-line definition
Hook	A function that lets a function component use state/other React features
useState	Gives a component memory that survives across re-renders
Rendering	React calls your component function to build a new virtual DOM description
Reconciliation	React compares the new virtual DOM to the old one to find differences
Commit	React applies the found differences to the real DOM
Scheduling	React decides when/in what priority order to do render work
Fiber	A JS object representing one unit of work, enabling pausable/resumable rendering
type	Tells React what kind of element this is — same type is reused, different type is recreated

key

Gives list items a stable identity
so reordering doesn't confuse
React
